



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

A Survey of Machine Scheduling Problems with Blocking and No-Wait in Process

Nicholas G. Hall, Chelliah Sriskandarajah,

To cite this article:

Nicholas G. Hall, Chelliah Sriskandarajah, (1996) A Survey of Machine Scheduling Problems with Blocking and No-Wait in Process. Operations Research 44(3):510-525. <https://doi.org/10.1287/opre.44.3.510>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 1996 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

A SURVEY OF MACHINE SCHEDULING PROBLEMS WITH BLOCKING AND NO-WAIT IN PROCESS

NICHOLAS G. HALL

The Ohio State University

CHELLIAH SRISKANDARAJAH

University of Toronto

(Received August 1991; revisions received July 1992, March 1993, February 1994; accepted March 1994)

An important class of machine scheduling problems is characterized by a no-wait or blocking production environment, where there is no intermediate buffer between machines. In a no-wait environment, a job must be processed from start to completion, without any interruption either on or between machines. Blocking occurs when a job, having completed processing on a machine, remains on the machine until a downstream machine becomes available for processing. A no-wait or blocking production environment typically arises from characteristics of the processing technology itself, or from the absence of storage capacity between operations of a job. In this review paper, we describe several well-documented applications of no-wait and blocking scheduling models and illustrate some ways in which the increasing use of modern manufacturing methods gives rise to other applications. We review the computational complexity of a wide variety of no-wait and blocking scheduling problems and describe several problems which remain open as to complexity. We study several deterministic flowshop, jobshop, and openshop problems and describe efficient and enumerative algorithms, as well as heuristics and results about their performance. The literature on stochastic no-wait and blocking scheduling problems is also reviewed. Finally, we provide some suggestions for future research directions.

In recent years, a considerable amount of interest has arisen in *no-wait* and *blocking* scheduling problems. This interest appears to be motivated both by applications and by research issues. A no-wait scheduling problem occurs in a production environment in which a job (or product, or part) must be processed from start to finish, without any interruption either on or between machines. Equivalently, the difference between the completion time of the last operation of a job and the start time of its first operation is equal to the sum of the processing times of its operations. Scheduling problems with blocking arise in serial manufacturing processes where no intermediate buffer storage is available. In such situations a job which has completed processing on a machine may remain there until a downstream machine becomes available, but this prevents another job from being processed there.

A variety of industrial applications has created recent interest in the modeling and methodologies of no-wait and blocking scheduling. For example, several papers dating back to the late 1970s address issues of computational complexity. More recently, several heuristic approaches have been described and analyzed. We are not aware of any attempt to summarize and synthesize the applied and theoretical studies on this topic. In view of the increasing importance of modern manufacturing issues relating to no-wait and blocking scheduling, we attempt to do so here. Our primary objectives are to review the applications of no-wait and blocking scheduling and the literature on related research problems, and to provide insights about unresolved issues and future research directions.

1. APPLICATIONS

1.1. Present Applications

The first of two main reasons for the occurrence of a no-wait or blocking production environment lies in the production technology itself. In some processes, for example, the temperature or other characteristics (such as viscosity) of the material require that each operation follow the previous one immediately.

Such situations arise in the production of steel, where molten steel undergoes a series of operations such as molding into ingots, unmolding, reheating, soaking, and preliminary rolling. Similarly, in the plastic molding and silverware production industries, a series of processes must follow one another immediately to prevent degradation. Further examples occur in the chemical and pharmaceutical industries. For similar reasons, the anodizing of aluminum products such as pipes, trim, and truck grilles has to be performed as a no-wait process. Similarly, in the food processing industry, the canning operation must immediately follow cooking to ensure freshness. Modern manufacturing environments such as just-in-time, flexible manufacturing systems, and robotic cells provide a highly coordinated manufacturing process which can frequently be modeled as a no-wait scheduling problem. Indeed, traditional scheduling models fail to address the need to meet demand requirements while simultaneously minimizing inventory. Finally, a no-wait environment occurs in service industries, where the customer has a prohibitively high cost of waiting in process.

Subject classifications: Production/scheduling: no-wait production environment. Manufacturing, automated systems: review of complexity, algorithms and heuristics
Area of review: MANUFACTURING, OPERATIONS, AND SCHEDULING.

The second reason for the occurrence of a no-wait or blocking production environment is a lack of storage (or buffer) capacity between intermediate machines or work stations. Two different environments arise here. In "no-wait" scheduling, a job must leave a machine immediately after processing is completed. The less restrictive case known as "blocking" permits a job to remain on a machine after processing if the next machine is busy, but no other job can be processed on that machine while it does so. In-line robotic cells, in which robots are located along an in-line conveyor or other transport system, as used for spot-welding and assembling car bodies, generate a no-wait scheduling environment. Just-in-time production lines generate the blocking environment, because the Kanban control system maintains a fixed limit on in-process inventory. A scheduling problem with blocking may be thought of as a relaxation of a similar scheduling problem with the no-wait assumption.

On the boundary of the no-wait/blocking scheduling environment are a number of interesting issues. For example, a flowshop problem with finite buffer capacity between operations is a very common scheduling environment. Much can be learned about this problem by studying the blocking environment, which is an extreme case of the finite capacity problem. As another example, the study of the blocking and no-wait environments has relevance to just-in-time systems and robotic cells. Furthermore, there are many industrial settings (for example, the steel industry) where the processing time of an operation is an increasing function of the wait since the previous operation. The no-wait scheduling environment is equivalent to a situation where this function increases very rapidly.

Useful general references on machine scheduling problems include the books by Conway et al. (1967), Baker (1974), Coffman (1976), and French (1982). Influential survey articles have been written by Gonzalez (1977), Graham et al. (1979), Lawler (1982), Lawler et al. (1982), Lenstra and Rinnooy Kan (1985), and Lawler et al. (1989).

Callahan (1971) describes a study of the steel industry, in which no-wait processing, or state (i.e., temperature) dependent processing, is required. A queueing model is used to analyze several problems, and computational testing is used to evaluate the recommendations.

Salvador (1973) describes an algorithm originally developed for a nylon polymerization production facility. The objective is to minimize the makespan for a set of n jobs, by determining an optimal sequence and an optimal set of start times for those jobs. Dynamic programming is used to find lower bounds for use in branch and bound. Implementation of the model substantially increased the throughput rate of the plant.

As noted by Reklaitis (1982), production of chemical products may require some batches to be produced as no-wait, and others with no intermediate storage. Egli and Rippin (1981) discuss such situations which arise from the chemical instability of intermediate products.

McCormick et al. (1989) study an m -machine cyclic flowshop with finite buffers, with the objective of minimizing cycle time, or equivalently maximizing throughput. Viewing the buffers as machines with zero processing times converts the problem into one with no buffers, but with blocking. The authors describe a heuristic approach which performs well.

While a major focus of modern manufacturing systems is the reduction, or even elimination, of inventory (Schonberger 1982, Zangwill 1987), from a scheduling perspective the presence of work-in-process inventory can be an advantage. As Conway et al. (1988) point out, workstations in series without intervening work-in-process must be perfectly synchronized to operate effectively. Matsuo (1990) points out that, in order to achieve the minimum cycle time in a flowshop producing several jobs repetitively, it may be necessary to allow work-in-process inventory. These results suggest that environments with limited buffer storage define constraints on scheduling decisions, as will become clear at several points in this paper.

On the boundary of the no-wait scheduling environment are the papers by Wagneur and Sriskandarajah (1990, 1993a,b), in which the processing time of an operation is dependent upon the wait since the finish time of the previous operation. This environment is motivated by temperature-dependent processing times in the steel industry.

Hsu and Stein (1996) consider a problem arising in a plant which manufactures aluminum products. Jobs with small processing times at early production stages relative to later stages are given priority over others. A significant improvement in the schedule length results from the implementation of this heuristic for instances with up to 150 jobs.

Logendran and Sriskandarajah (1996) study an in-line robotic cell with two robots, and show that finding a schedule of parts to maximize throughput can be solved by the algorithm of Gilmore and Gomory (1964), see Section 3.1. A similar result is proved by Kise et al. (1991) for schedule length minimization in an automated two-machine flowshop where jobs are transported between the two machines by a single automated guided vehicle.

Ramudhin and Ratliff (1992b) consider a no-wait scheduling problem in a plant manufacturing condiments such as mustard. Due to the absence of holding tanks, and to avoid blocking, each batch is processed without delays between the processing of two consecutive operations. A two-phase heuristic, based on mathematical programming principles, works interactively with an expert user.

Sethi et al. (1992) show, under a restrictive assumption, that the problem of sequencing parts to maximize throughput in a two-machine robotic cell is equivalent to minimizing cycle time in a two-machine flowshop producing several jobs repetitively, and thus is solvable by the efficient algorithm of Gilmore and Gomory (1964). In the three-machine robotic cell problem, it is not clear that the maximum throughput is given by a single routing between machines for the robot. Hall et al. (1996) provide a polynomial time algorithm to minimize a general version

of the two machine robotic cell throughput problem. They also show that, for four out of the six possible robot routings in the three machine cell, polynomial time algorithms exist. Hall et al. (1995) show that in the other two routings the part sequencing problem is *NP*-complete. Sriskandarajah et al. (1996) generalize these results to robotic cells with m machines, for any $m \geq 2$.

Dudek et al. (1992), in attempting to draw together the lessons to be learned from 38 years of flowshop scheduling research, point out that the only flowshop makespan problem they have seen in manufacturing is of the no-wait type described by Wismer (1972).

1.2. Future Applications

We expect future applications of no-wait scheduling to arise mainly in the area of modern manufacturing, especially because of automation of the materials handling process. Many just-in-time production systems can be modeled as flowshops with blocking. Since this problem is very hard, the development of more accurate procedures is a major challenge. Flexible manufacturing systems attempt to eliminate in-process storage space, and also have limited materials-handling capacity, two characteristics which suggest no-wait or blocking models as useful. Similarly, robotic cells frequently have no storage areas, creating a scheduling problem with the no-wait characteristic, or with blocking (Sethi et al. 1992, Kamoun et al. 1993, Hall et al. 1995, 1996).

2. COMPUTATIONAL COMPLEXITY

2.1. Background and Notation

The fundamental concepts of computational complexity are outlined in Garey and Johnson (1979). A variety of complexity results for machine scheduling problems are described in Rinnooy Kan (1976), Lenstra (1977), Lenstra et al. (1977), Lenstra and Rinnooy Kan (1979), and Lawler et al. (1989). Preliminary results for no-wait scheduling problems appear in Goyal and Sriskandarajah (1988). The recognition version of a machine scheduling problem is said to be *NP*-complete in the ordinary (respectively, strong) sense if the existence of a polynomial (resp., pseudo-polynomial) time algorithm for the equivalent decision problem would imply that $P = NP$.

In a *flowshop* all jobs follow the same machine sequence, and each job has exactly one operation on each machine. An *openshop* is similar to a flowshop, except that the operations of a job may be performed in any order. In a *jobshop* jobs may follow different machine sequences and may use the same machine more than once. Variations of these problems allow jobs to be processed on some, but not all, machines. Several identical machines, any one of which can process any job, form a parallel machine environment.

We use the following notation:

w_j = weight (value) of job j .
 d_j = due date of job j .

C_j = completion time of job j .

$U_j = \begin{cases} 1 & \text{if } C_j > d_j \\ 0 & \text{otherwise.} \end{cases}$

I_j = total idle time immediately preceding job j , summed over all machines.

$p_{i,j}$ = processing time of job j on machine i .

r_j = release time of job j .

P_m = m -machine parallel shop.

P = parallel machine shop, where the number of machines is a variable, the value of which is specified as part of the problem instance.

F_m = m -machine flowshop, where $m \geq 2$ and each job has exactly m operations.

F_2^q = 2-machine flowshop, where jobs have two or fewer operations.

F = flowshop, where the number of machines is a variable, the value of which is specified as part of the problem instance.

F^q = flowshop, where the number of machines is a variable, and each job has that number or fewer operations.

J_m = m -machine jobshop, where $m \geq 2$.

O_m = m -machine openshop, where $m \geq 2$ and each job has exactly m operations.

O = openshop, where the number of machines is a variable, the value of which is specified as part of the problem instance.

O^q = openshop, where the number of machines is a variable, and each job has that number or fewer operations.

S_2 = two-stage process shop in which each stage consists of a number of machines operating in parallel, and each job consists of two operations that require processing on one of a set of specified machines at each stage.

A_2 = two-stage assembly shop where in the first stage each of $m \geq 2$ machines processes part of a job, and in the second stage a single machine assembles all the parts.

$M = \{M_1, \dots, M_m\}$, the set of available machines.

$N = \{1, \dots, n\}$, the set of available jobs.

m_i = number of identical parallel

machines at stage i of the production process.

sdpt = a state (e.g., temperature) dependent processing time, an environment in which processing time at an operation depends upon waiting time since the previous operation.

pmtn = preemption of an operation of a job is permitted.

res xyz = a resource constrained environment with x different resources having available quantity y , and a resource requirement per operation no greater than z . Here x , y and z can equal either a positive integer, or “.”. in the latter case, the value 3 specified or part of the problem instance.

b = nonzero, finite buffer capacity between machines.

d = a common due date for all jobs.

blocking ($i, i + 1$) = a blocking environment exists between machines i and $i + 1$.

no-wait ($i, i + 1$) = a no-wait environment exists between machines i and $i + 1$.

prec = a precedence relation is given for the jobs.

in-tree = a precedence relation in which each job, except the last, has one successor.

out-tree = a precedence relation in which each job, except the first, has one predecessor.

tree = a precedence relation that is an in-tree or an out-tree.

separated = each operation of each job consists of setup, processing and removal stages. Note that any stage of an operation on a machine may be performed concurrently with the setup stage of an operation on a later machine.

controllable speed = an environment where machine speeds may be increased at a cost.

ranked data = an ordinal ranking for the data, e.g., for the processing times, but not exact values, is known.

The objective function is variously defined as the minimization of:

$C_{\max} = \max_{1 \leq j \leq n} \{C_j\}$, the finish time of the last job (makespan).

$L_{\max} = \max_{1 \leq j \leq n} \{C_j - d_j\}$, the maximum lateness.

$\sum_{j=1}^n C_j$ = the total (or equivalently the mean) flow time.

$\sum_{j=1}^n w_j U_j$ = the total weight of late jobs.

$\sum_{j=1}^n w_j I_j$ = the total weighted idle time.

C_i = the cycle time in a machine shop producing a given set of jobs repetitively.

W_i = the total waiting time of work-in-process jobs per cycle.

Two variations of the no-wait flowshop model, F_m and F_m^g , are identified (Hefetz and Adiri 1982). In F_m^g , each job has m or fewer operations, whereas in F_m each job is processed on all the machines. Similarly we have two openshop models, O_m and O_m^g .

2.2. Summary of Known Results

We follow the standard notation for machine scheduling problems suggested by Graham et al. (1979). The classification $\alpha|\beta|\gamma$ is used, where α denotes the machine configuration, β indicates the job characteristics or special constraints, and γ defines the optimality criterion. Thus, for example, $F_2|\text{no-wait}|C_{\max}$ denotes the minimization of maximum completion time (i.e., makespan) in a two-machine no-wait flowshop. The notation for resource constraints was proposed by Blazewicz et al. (1983). The known computational complexity results for no-wait scheduling problems are summarized in Table I. A number of the results in Table I depend upon the algorithm of Gilmore and Gomory, as described in Section 3.1.

The results shown in Table I describe the most specific version of a problem for which an *NP*-completeness result is known, or the most general version of a problem for which a given algorithm is available. In this way, we describe as accurately as possible the boundary between easy and hard problems.

A few comments about Table I now follow. First, no-wait scheduling with several machines is always at least as difficult as with a single machine, whatever the configuration of the machines. Thus we can infer from the results in Sahni and Cho (1979) and Kubiak et al. (1991) the strong *NP*-completeness of $O_2|\text{no-wait}, m_1 \geq 1, m_2 \geq 1|C_{\max}$ and $O_2|\text{no-wait}, m_1 \geq 1, m_2 \geq 1|\sum C_j$, respectively. Recall also that the jobshop configuration is a generalization of the flowshop, therefore any *NP*-completeness result which is known for a flowshop can immediately be transferred to a similar result for a jobshop.

2.3. Linkages Between Problems

The following lemmas enable us to provide some of the results in Table I.

Lemma 1. $F_3|\text{blocking}|C_{\max}$ is strongly *NP*-complete.

Proof. Papadimitriou and Kanellakis (1980) prove the strong *NP*-completeness of $F_2|b = 1|C_{\max}$. This problem is equivalent to a special case of $F_3|\text{blocking}|C_{\max}$ in which the second machine has zero processing times for all jobs.

Lemma 2. $F_3|\text{blocking} (1,2), \text{no-wait} (2,3)|C_{\max}$ is strongly *NP*-complete.

Table I
Complexity of Blocking and No-wait Scheduling Problems

Problem	Complexity	Reference	Comments
$F_2 \text{no-wait} \gamma$	P	Gilmore and Gomory (1964)	$O(n \log n)$ time for $\gamma \in \{C_{\max}, C_t\}$. Similarly for $F_2 \text{blocking} \gamma$.
$F_2 \text{sdpt} C_{\max}$	CS	Sriskandarajah and Wagneur (1991)	Processing time depends upon waiting time.
$F_2 \text{no-wait}, m_1 = 1, m_2 = 2 C_{\max}$	CS	Sriskandarajah and Ladet (1986)	Similarly if $m_1 = 2, m_2 = 1$. Similarly for blocking.
$F_2 \text{blocking}, m_1 = 1, m_2 = 2 C_t$	CS	Kamoun and Sriskandarajah (1993)	Similarly if $m_1 = 2, m_2 = 1$. Similarly for no-wait.
$F_2 \text{blocking, separated} C_{\max}$	CS	Logendran and Sriskandarajah (1993)	Result holds even with zero removal times. Strusevich (1990) provides an $O(n \log n)$ time algorithm for the no-wait case.
$F_2 \text{no-wait} L_{\max}$	CS	Röck (1984b)	Similarly for $F_2 \text{no-wait}, r_j \gamma, \gamma \in \{C_{\max}, C_t\}$.
$F_2 \text{no-wait} \Sigma C_j$	CS	Röck (1984b)	Solvable in $O(n \log n)$ time even for m machines for “semi-ordered” processing time matrix (van der Veen and van Dal 1991).
$F_2 \text{minimum } C_t W_t$	CS	Matsuo (1990)	Similarly for C_t objective. $b = +\infty$ case is solvable in $O(n \log n)$ time (Johnson 1954) for $C_{\max}$ objective, trivial for C_t objective.
$F_2 b C_{\max}$	CS	Papadimitriou and Kanellakis (1980)	
$F_2 \text{no-wait, res11} C_{\max}$	Open	Röck (1984b)	Similarly for C_t objective.
$F_2 \text{no-wait}, p_{i,j} = 1, \text{res1..} C_{\max}$	P		$O(n \log n)$ time solution via Gilmore and Gomory (1964) algorithm. Similarly for C_t objective.
$F_2 \text{no-wait}, p_{i,j} = 1, \text{res1..} L_{\max}$	CS	Röck (1984b)	Similarly for $F_2 \text{no-wait}, p_{i,j} = 1, \text{res1..}, r_j \gamma, \gamma \in \{C_{\max}, C_t\}$.
$F_2 \text{no-wait}, p_{i,j} = 1, \text{res2..} C_{\max}$	CS	Röck (1983)	Similarly for C_t objective.
$F_2 \text{no-wait}, p_{i,j} = 1, \text{res1..} \Sigma C_j$	CS	Röck (1983)	
$F_2 \text{no-wait}, p_{i,j} = 1, \text{res11} L_{\max}$	Open		See Blazewicz et al. (1988) for a related result.
$F_2^g \text{no-wait} C_{\max}$	CS	Sahni and Cho (1979)	Similarly for C_t objective.
$F_3 \text{no-wait}, p_{i,j} = 1, \text{res1..} C_{\max}$	CS	Röck (1984b)	Similarly for C_t objective.
$F_3 \text{no-wait} C_{\max}$	CS	Röck (1984a)	Similarly for C_t objective. Solvable in $O(n^2)$ time even for m machines for “semi-ordered” processing time matrix (Arora and Rana, 1980; Axsäter, 1982).
$F_3 \text{no-wait}(1,2) C_{\max}$	CS	Röck (1984b)	No-wait requirement for the first two machines only. Similarly for C_t objective and for blocking (1,2) with either objective, and for no-wait (2,3) or blocking (2,3).
$F_3 \text{blocking} C_{\max}$	CS	See Lemma 1	Follows from Papadimitriou and Kanellakis (1980). Similarly for $F_3 \text{blocking} C_t$ (Kamoun and Sriskandarajah, 1993).
$F_3 \text{blocking}(1,2), \text{no-wait}(2,3) C_{\max}$	CS	See Lemma 2	Similarly for C_t objective (Kamoun and Sriskandarajah, 1993).
$F_3 \text{no-wait}(1,2), \text{blocking}(2,3) C_{\max}$	CS	See Lemma 3	Similarly for C_t objective (Kamoun and Sriskandarajah, 1993).
$F_m \text{no-wait}, p_{i,j} = 1, \text{res1.1} C_{\max}$	Open		See Blazewicz et al. (1988) for a related result. Similarly for C_t objective.
$F \text{no-wait}, p_{i,j} = 1, \text{res111} C_{\max}$	CS	Blazewicz et al. (1988)	Solvable in $O(n \log n)$ time if $m = 3$ (Sriskandarajah and Ladet 1986).
$F^g \text{blocking}, p_{i,j} \in \{0, 1\} C_t$	CS	McCormick et al. (1987)	
$F^g \text{no-wait}, p_{i,j} \in \{0, 1\} C_{\max}$	CS	Gonzalez (1982)	
$F^g \text{no-wait}, p_{i,j} \in \{0, 1\} \Sigma C_j$	CS	Gonzalez (1982)	Solvable in $O(n \log n)$ time if $m = 2$ (Sriskandarajah and Ladet 1986).
$J_2 \text{no-wait} C_{\max}$	CS	Sahni and Cho (1979)	Even if each job has exactly two nonzero operations. Similarly for blocking and C_t objective (Kamoun and Sriskandarajah 1993).

Table I
(Continued)

Problem	Complexity	Reference	Comments
$J_2 \text{no-wait}, p_{i,j} = 1 C_{\max}$	CO, PP	Sriskandarajah and Ladet (1986), Kubiak (1989)	$O(n^5k)$ time, where K = the total number of operations of the jobs.
$J_3 \text{no-wait}, p_{i,j} = 1 C_{\max}$	CS	Sriskandarajah and Ladet (1986)	
$J_3 \text{no-wait}, p_{i,j} = 1 \sum C_j$	CS	Sriskandarajah and Ladet (1986)	
$O_2 \text{no-wait} C_{\max}$	CS	Sahni and Cho (1979)	Similarly for blocking and for C_i objective (Kamoun and Sriskandarajah 1993). Without the no-wait restriction, the problem $O_2 C_{\max}$ is solvable in $O(n)$ time (Gonzalez and Sahni 1976), and $O_2 C_i$ is trivial.
$O_2 \text{no-wait}, \text{pmtn} C_{\max}$	CS	Strusevich (1991)	An operation can be preempted for an interval, during which another operation must be continuously processed.
$O_2 \text{no-wait}, \text{prec}, p_{i,j} = 1 C_{\max}$	P	Coffman and Graham (1972), Lawler (1976), Brucker et al. (1993)	$O(n^2)$ time.
$O_2 \text{no-wait}, \text{prec}, p_{i,j} = 1 L_{\max}$	P	Garey and Johnson (1976), Brucker et al. (1993)	$O(n^3 \log n)$ time.
$O_2 \text{no-wait}, r_j, \text{prec}, p_{i,j} = 1 L_{\max}$	P	Garey and Johnson (1977), Brucker et al. (1993)	$O(n^3)$ time.
$O_2 \text{minimum } C_i W_i$	CS	Kamoun and Sriskandarajah (1993)	
$O_2 \text{no-wait} \sum C_j$	CS	Kubiak et al. (1991)	
$O \text{no-wait}, p_{i,j} = 1 \sum C_j$	P	Adiri and Amit (1984)	$O(mn)$ time.
$O \text{no-wait}, \text{prec}, p_{i,j} = 1 \sum C_j$	CS	Lenstra and Rinnooy Kan (1978), Brucker et al. (1993)	
$O \text{no-wait}, \text{out-tree}, p_{i,j} = 1 \sum C_j$	P	Hu (1961), Sethi (1976), Brucker et al. (1993)	$O(n)$ time.
$O \text{no-wait}, p_{i,j} = 1 \sum U_j$	P	Lawler (1976), Brucker et al. (1993)	$O(n \log n)$ time.
$O \text{no-wait}, p_{i,j} = 1 \sum w_j U_j$	P	Kubiak et al. (1991)	$O(n \log n)$ time.
$O \text{no-wait}, p_{i,j} = 1 \sum w_j T_j$	P	Brucker et al. (1993)	$O(n^3)$ time.
$O \text{no-wait}, \text{prec}, p_{i,j} = 1 C_{\max}$	CS	Ullman (1975), Brucker et al. (1993)	
$O \text{no-wait}, r_j, p_{i,j} = 1 C_{\max}$	P	Jackson (1955), Brucker et al. (1993)	$O(n \log n)$ time.
$O \text{no-wait}, \text{tree}, p_{i,j} = 1 C_{\max}$	P	Hu (1961), Sethi (1976), Brucker et al. (1993)	$O(n)$ time.
$O \text{no-wait}, \text{out-tree}, p_{i,j} = 1 L_{\max}$	CS	Brucker et al. (1977), Brucker et al. (1993)	
$O \text{no-wait}, r_j, p_{i,j} = 1 L_{\max}$	P	Simons and Warmuth (1989), Brucker et al. (1993)	$O(mn^2)$ time.
$O \text{no-wait}, \text{in-tree}, p_{i,j} = 1 L_{\max}$	P	Brucker et al. (1977), Monma (1979), Brucker et al. (1993)	$O(n)$ time.
$O^g \text{no-wait}, p_{i,j} \in \{0, 1\} C_{\max}$	CS	Gonzalez (1982)	Similarly for $\sum C_i$ objective.
$S_2 \text{no-wait} C_{\max}$	CS	Ramudhin and Ratliff (1992a)	Solvable by Gilmore and Gomory (1964) in $O(n \log n)$ time if only one machine is available at both stages. Strongly NP-complete if either stage has two or more machines in parallel.
$A_2 \text{no-wait}, \text{separated} C_{\max}$	CS	Strusevich and Zwaneveld (1991)	Solvable by Gilmore and Gomory (1964) in $O(n \log n)$ time if either the removal times, or the sum of setup and processing times, in the first stage, is machine-independent.

Complexity notation:

P = a polynomial time algorithm exists.

PP = a pseudo-polynomial time algorithm exists.

CO = the equivalent decision problem is NP-complete in the ordinary sense.

CS = the equivalent decision problem is NP-complete in the strong sense.

Proof. Röck (1984a) proves the strong NP -completeness of $F_3|no\text{-}wait|C_{\max}$. Any blocking schedule between machines 1 and 2 can be transformed into a no-wait schedule, and vice versa, without increasing or decreasing the objective function value.

Lemma 3. $F_3|no\text{-}wait (1,2), blocking (2,3)|C_{\max}$ is strongly NP -complete.

Proof. Lemma 1 proves the strong NP -completeness of $F_3|blocking|C_{\max}$. Any schedule for $F_3|no\text{-}wait (1,2), blocking (2,3)|C_{\max}$ can be transformed into a schedule for $F_3|blocking|C_{\max}$, and vice versa, without decreasing or increasing the value of $C_{\max}$.

Lemma 4. $\alpha_2|blocking, m_1 \geq 1, m_2 \geq 1|\gamma$ is equivalent to $\alpha_2|no\text{-}wait, m_1 \geq 1, m_2 \geq 1|\gamma$ for $\alpha \in \{F, O\}$, $\gamma =$ any regular performance measure, and nonseparated processing times.

Proof. A regular performance measure is any that is non-decreasing in the job completion times. The nonseparated processing times mean that the setup times and removal times of each operation are either zero or cannot be separated from the processing time. Given a schedule for the blocking problem, we can easily construct a schedule for the no-wait problem with the same objective function value. Since the blocking problem is a relaxation of the no-wait problem, the equivalence of the two problems follows.

Below we describe algorithms and heuristics for no-wait and blocking scheduling problems. In some cases, we assess the performance of a heuristic according to its worst case performance bound. Let f (respectively, f^*) denote the value of a heuristic (resp., optimal) solution. If a constant $r > 1$ can be found such that $f/f^* \leq r$ for any problem instance, then r defines the worst case performance bound of the heuristic. We also review some enumerative studies and comment upon their computational performance.

3. DETERMINISTIC FLOWSHOP PROBLEMS

Much of the work on no-wait and blocking scheduling problems has been written about the flowshop environment, which we now consider.

3.1. $F_2|no\text{-}wait|C_t$

Here a given set of jobs is produced repetitively, once each cycle. The algorithm described below solves this problem. The intuition behind the algorithm is that, ideally, the shortest processing time on machine 1 would be concurrent with that on machine 2, similarly for the second shortest processing times on the two machines, and so on. If this is not possible, a dual improvement step moves the current schedule towards feasibility at minimum cost. Let $\Psi(j)$ denote the job that follows job j in the sequence found.

Algorithm (Gilmore and Gomory 1964)

STEP 1. Number the jobs such that $p_{2,j} \leq p_{2,j+1}$, $j = 1, \dots, n-1$. Initialize $G_1 = G_2 = \emptyset$.

STEP 2. Find a function $\phi(j)$, $j = 1, \dots, n$, such that $p_{1,\phi(j)} \leq p_{1,\phi(j+1)}$, $j = 1, \dots, n-1$.

STEP 3. Define a graph with n nodes (each representing a job) and no edges. The lengths $C_{j,j+1}$ of edges $(j, j+1)$, $j = 1, \dots, n-1$ that may be added later are given by $C_{j,j+1} = \max\{0, (\min\{p_{2,j+1}, p_{1,\phi(j+1)}\} - \max\{p_{2,j}, p_{1,\phi(j)}\})\}$ for $j = 1, \dots, n-1$.

STEP 4. Set $j = 1$.

STEP 4.1. If the undirected edge $(j, \phi(j))$ is not in the graph and $j \neq \phi(j)$, add it. Set $j = j+1$.

STEP 4.2. If $j \leq n$, go to Step 4.1.

STEP 5. If the graph has only one connected component, go to Step 7. Otherwise, let $k = \operatorname{argmin}\{C_{j,j+1} | j \text{ and } j+1 \text{ are in different components}\}$, breaking ties arbitrarily.

STEP 6. Add the undirected edge $(k, k+1)$ to the graph. If $p_{1,\phi(k)} \geq p_{2,k}$, set $G_1 = G_1 \cup \{(k, k+1)\}$. Otherwise set $G_2 = G_2 \cup \{(k, k+1)\}$. Go to Step 5.

STEP 7. If $G_1 = \emptyset$, let $s = 0$. Otherwise, let the elements of G_1 be $\{(r_1, r_1+1), \dots, (r_s, r_s+1)\}$, where $r_1 \geq \dots \geq r_s$.

STEP 7.1. If $G_2 = \emptyset$, let $t = 0$. Otherwise, let the elements of G_2 be $\{(k_1, k_1+1), \dots, (k_t, k_t+1)\}$, where $k_1 \leq \dots \leq k_t$.

STEP 8. Define for $1 \leq e, g, h \leq n$ a function $\alpha_{e,g}(h)$ as follows: $\alpha_{e,g}(e) = g$, $\alpha_{e,g}(g) = e$, and $\alpha_{e,g}(h) = h$ if $h \neq e$, g . Set $j = 1$.

STEP 8.1. If $t = s = 0$, set $\Psi(k) = \phi(k)$, $k = 1, \dots, n$, and stop.

STEP 8.2. Set $y = j$. If $t = 0$, set $i = s$ and go to Step 8.5. Otherwise, set $i = t$.

STEP 8.3. Set $y = \alpha_{k_i, k_i+1}(y)$ and $i = i-1$.

STEP 8.4. If $i \geq 1$, go to Step 8.3. Otherwise, if $s = 0$, go to Step 8.6. Otherwise, set $i = s$.

STEP 8.5. Set $y = \alpha_{r_i, r_i+1}(y)$ and $i = i-1$.

STEP 8.6. If $i \geq 1$, go to Step 8.5. Otherwise, set $\Psi(j) = \phi(y)$ and $j = j+1$.

STEP 8.7. If $j \leq n$, go to Step 8.2. Otherwise, stop.

This algorithm can be implemented in $O(n \log n)$ time. Now consider the following example.

Example 3.1.

Job j	1	2	3	4	5	6	7	8
$p_{1,j}$	10	12	3	5	6	11	9	4
$p_{2,j}$	7	8	2	3	9	12	13	6

Step 1 gives:

Job j	1	2	3	4	5	6	7	8
$p_{2,j}$	2	3	6	7	8	9	12	13
$p_{1,j}$	3	5	4	10	12	6	11	9

Steps 2 and 3 give:

Job j	$p_{2,j}$	$p_{1,\phi(j)}$	$\phi(j)$	$\max\{p_{2,j}, p_{1,\phi(j)}\}$	$\min\{p_{2,j}, p_{1,\phi(j)}\}$	$C_{j,j+1}$
1	2	3	1	3	2	0
2	3	4	3	4	3	1
3	6	5	2	6	5	0
4	7	6	6	7	6	1
5	8	9	8	9	8	0
6	9	10	4	10	9	1
7	12	11	7	12	11	0
8	13	12	5	13	12	-

At Step 4, the edges in the graph are (2,3), (4,6), and (5,8).

At Step 5, the graph has components $\{1\}$, $\{2,3\}$, $\{4,6\}$, $\{5,8\}$, and $\{7\}$.

At Step 6, edges (1,2), (3,4), (5,6), and (7,8) are added, $G_1 = \{(1,2), (5,6)\}$, $G_2 = \{(3,4), (7,8)\}$.

At Step 7, $r_1 = 5$, $r_2 = 1$, $k_1 = 3$, $k_2 = 7$.

At Step 8, for $j = 1$ we have: $y = \alpha_{7,8}(1) = 1$, $\alpha_{3,4}(1) = 1$, $\alpha_{1,2}(1) = 2$, $\alpha_{5,6}(2) = 2$, $\Psi(1) = \phi(2) = 3$. The steps are similar for $j = 2, \dots, 8$.

Thus the optimal sequence, with a cycle time of 64, is given by:

Job j	1	2	3	4	5	6	7	8
$\Psi^*(j)$	3	1	6	2	4	8	5	7

The optimal makespan of job set J in $F_2|no-wait|C_{max}$ is equal to the optimal cycle time of job set $J \cup \{j_0\}$ in $F_2|no-wait|C_r$, where j_0 is an artificial job with zero processing time on both machines. Therefore the Gilmore-Gomory algorithm also solves $F_2|no-wait|C_{max}$.

Agnetis (1989) considers a variant of this problem in which the n jobs can be grouped into s lots. Within a lot, all jobs have identical processing times on all machines. This permits implementation of the Gilmore-Gomory algorithm in $O(s \log s + \min\{s^2, n\})$ time. This result also holds with blocking, from Lemma 4.

Strusevich (1990) considers a generalization of $F_2|no-wait|C_{max}$ in which the time spent by each job on each machine is divided into a setup time, a processing time, and a removal time (e.g., for disengaging and removing tools). A job setup on machine 2 can overlap its processing on machine 1. The problem is converted into a special case of the traveling salesman problem, solvable by the algorithm of Gilmore and Gomory. However, the blocking

version of this problem is shown to be strongly NP -complete by Logendran and Sriskandarajah (1993).

3.2. F_2 Boundary Problems

Several problems define a continuum starting from the $F_2|no-wait|C_{max}$ problem considered in Section 3.1. That is, consider a two-machine flowshop problem with an intermediate storage buffer of capacity b , or $F_2|b|C_{max}$. If $b = 0$, the algorithm of Gilmore and Gomory is appropriate, and if $b \geq n - 1$, the algorithm of Johnson (1954) finds optimal solutions in $O(n \log n)$ time. Jackson (1956) extends this result to the environment of a 2-machine jobshop where each job has no more than two operations. Clearly a knowledge of these algorithms may suggest procedures for the case $0 < b < n - 1$, as we discuss below. Papadimitriou and Kanellakis show that, for any $1 \leq b < n - 1$, $F_2|b|C_{max}$ is strongly NP -complete, and describe a heuristic for the case $b = 1$. This heuristic first finds an optimal sequence of jobs in $F_2|no-wait|C_{max}$ using the algorithm of Gilmore and Gomory, then uses that sequence to schedule all the jobs in $F_2|b = 1|C_{max}$. The authors establish a tight worst case bound of $r = 3/2$ for this heuristic. This problem has also been studied by Dutta and Cunningham (1975), who describe a dynamic programming algorithm. This algorithm has an exponential state space which defines the set of jobs already processed, requiring excessive storage for instances larger than $n = 18$. The authors also describe a limited enumeration approach which for $n = 20$ routinely provides solutions within 5% of optimal in less than one second of CPU time.

Leisten (1990) undertakes a computational study which compares several heuristics previously published for the unlimited buffer case, the heuristic of King and Spachis (1980), and two specially designed heuristics which attempt to optimize the utilization of the available buffer storage. The heuristic which performed best in computational testing was earlier used by Nawaz et al. (1983). Since a major complicating issue in this problem is the finite buffer capacity, we believe that techniques from mathematical programming which involve relaxing that constraint would have considerable computational power.

Also on the boundary of the no-wait scheduling environment are problems with state dependent processing times. Sriskandarajah and Wagneur (1991) study $F_2|sdpt|C_{max}$, where the processing time of a job on its next operation depends upon the waiting time of that job in the buffer between the machines. They show that the problem is strongly NP -complete. Surprisingly, the optimal schedule may not be a permutation schedule (Wagneur and Sriskandarajah 1993a). Some performance bounds for permutation schedule heuristics are developed in Sriskandarajah and Goyal (1989). Wagneur and Sriskandarajah (1990) describe a branch-and-bound algorithm capable of solving instances of the m -machine version of this problem up to $m = 4$ and $n = 12$.

3.3. $F_2|no\text{-}wait, p_{ij} = 1, res\ 1..|C_{max}$

In this problem, each job j requires processing on both M_1 and M_2 for $p_{1,j} = 1, p_{2,j} = 1$. Röck (1984b) suggests an $O(n \log n)$ time algorithm that adds dummy jobs, and makes use of the algorithm of Gilmore and Gomory. The algorithm can also be used to solve the corresponding problem with C_t objective.

3.4. $F_2|no\text{-}wait, controllable\ speed|total\ cost$

Strusevich (1995) considers a two-machine no-wait flowshop in which machine speeds (v_1, v_2) are controllable, and minimizes the total cost function $k_0 t(v_1, v_2)^{k_3} + k_1 v_1^{k_4} + k_2 v_2^{k_4}$, where k_0, k_1, k_2 are given positive constants, k_3 and k_4 are given positive integers, and $t(v_1, v_2)$ denotes the minimum makespan value achievable with machine speeds v_1 and v_2 . The algorithm starts by finding the $O(n^2)$ values for the ratio v_1/v_2 at which the optimum permutation of jobs might change. At each of these values, several of the steps in the algorithm of Gilmore and Gomory (1964) which require only $O(n)$ time are used to find an optimum permutation in $O(n^3)$ time. This creates a candidate set of solutions which can be compared in $O(n^2)$ time, so the overall complexity of the algorithm is $O(n^3)$. This result also holds with blocking, from Lemma 4.

3.5. Flowshops with Unit Processing Times

A simple algorithm for $F_2|no\text{-}wait, p_{ij} \in \{0, 1\}|\Sigma C_j$ is described by Sriskandarajah and Ladet (1986). Let J_{xy} denote the type of job with nonzero processing time on machines x and y . The algorithm first schedules J_1 type jobs and J_2 type jobs on M_1 and M_2 , respectively, then J_{12} type jobs at the end of the schedule. It is easy to see that the algorithm is optimal with

$$\sum C_j = \frac{(n_1 + 1)n_1}{2} + \frac{(n_2 + 1)n_2}{2} + \frac{(n_{12} + 1)n_{12}}{2} + n_0 n_{12},$$

where n_1, n_2, n_{12} are the number of jobs of J_1, J_2, J_{12} types, respectively, and $n_0 = \max\{n_1 + 1, n_2\}$.

A similar approach can be used for $F_3|no\text{-}wait, p_{ij} \in \{0, 1\}|C_{max}$. Extending the above notation, let J_{xyz} denote the type of job with nonzero processing time on machines x, y , and z . There are seven possible types of job: $J_{123}, J_{12}, J_{13}, J_{23}, J_1, J_2$, and J_3 . The last three types are easy to include, following the scheduling of the first four types. An algorithm is presented in Sriskandarajah and Ladet.

3.6. $F_2|minimum\ C_t|W_t$

The objective here is to minimize total waiting time, subject to maintaining the minimum possible cycle time (Hall 1988). Matsuo (1990) develops a heuristic by using the sequence of jobs found by the algorithm of Gilmore and Gomory and dispatching the next job in that sequence when a machine becomes available. The (attainable) worst case bound of this heuristic is $(n - 1)$. If processing times of jobs are generated from a uniform $[0, 1]$ distribution, the

average waiting time of a job is no greater than $0.23\sqrt{n} + 1$. An important insight of Matsuo's work is that it simultaneously addresses the issues of cycle time minimization, or equivalently throughput rate maximization, and inventory control.

3.7. Flowshops with Parallel Machines

In the flowshop with parallel machines, each stage of processing, i , has $m_i \geq 1$ identical, parallel machines. Kuriyan (1987) considers the problem of minimizing makespan in a special case where $p_{1,j} = \dots = p_{m,j}$ and $m_i = \mu, i = 1, \dots, m$, and shows that a list scheduling heuristic provides a worst-case performance bound of $2 - 1/\mu$. If an LPT list is used, the bound improves to

$$1 + \left(\frac{\mu - 1}{\mu}\right)\left(\frac{m - 1}{m + 1}\right).$$

As a result of extensive computational testing, Kuriyan and Reklaitis (1985) conclude that the most effective of a number of heuristics is scheduling jobs in LPT order, supplemented by neighborhood search. For a two-stage model, Kuriyan and Reklaitis (1989) find that two local search heuristics, "best-fit" and "bottleneck-sequencing," provide the best computational results. It would be interesting to study the performance of a tabu search heuristic (Glover 1990) instead of neighborhood search.

Ramudhin and Ratliff (1992b) consider the problem of maximizing the weighted sum of customer orders that can be completed within an eight hour shift in a two-stage parallel flowshop. This problem can be denoted $F_2|no\text{-}wait, d_j = d, m_1 \geq 1, m_2 \geq 1|\Sigma w_j U_j$. They formulate the problem as a mathematical program, and use Lagrangean relaxation to obtain a decomposition into several subproblems, each of which involves finding the longest path in an acyclic network. Some or all infeasibilities of the resulting schedule are removed by a local improvement heuristic, then an expert user works interactively with the schedule.

Sriskandarajah (1993) establishes a tight (attainable) worst case bound of $r = 3 - 1/\mu$ using an arbitrary sequence for the list scheduling algorithm for $F_2|no\text{-}wait, m_1 = 1, m_2 = \mu \geq 2|C_{max}$. In this algorithm, jobs are scheduled in the order in which they appear in a randomly generated list. If the jobs are ordered by nonincreasing $p_{2,j}$ values, then a better result can be obtained. Sriskandarajah establishes the (attainable) bound $r = 2$ for this case.

Note that in the $F_2|no\text{-}wait, m_1 = 1, m_2 = \mu \geq 2|C_{max}$ problem, stage 1 has one machine, and stage 2 has μ parallel machines. An algorithm H for that problem can be applied to $F_2|no\text{-}wait, m_1 = \mu \geq 2, m_2 = 1|C_{max}$ as follows: interchange the values of $p_{1,j}$ and $p_{2,j}$ for each $j = 1, \dots, n$, in the latter problem. Then jobs are scheduled for the former problem using algorithm H with newly assigned values of $p_{1,j}, p_{2,j}, j = 1, \dots, n$. If we interchange the rôles of stages 1 and 2, and consider the schedule in the reverse direction, this will form a valid schedule for the latter problem. Thus, we have the following result.

Lemma 5. *Algorithm H for $F_2|no\text{-}wait, m_1 = 1, m_2 = \mu \geq 2|C_{max}$ when applied to $F_2|no\text{-}wait, m_1 = \mu \geq 2, m_2 = 1|C_{max}$ as above, has the same worst case performance bound.*

It follows that the above two algorithms, when applied to $F_2|no\text{-}wait, m_1 = \mu \geq 2, m_2 = 1|C_{max}$ as above, have worst case bounds of $3 - 1/\mu$ and 2, respectively. Further, as a consequence of Lemma 4, all the above results hold if the blocking constraint is imposed instead of no-wait. Performance bounds for the above heuristics for $F_2|no\text{-}wait, m_1 \geq 2, m_2 \geq 2|C_{max}$ have not been established. It would be interesting to pursue such research.

Knopf (1985) considers the minimization of makespan in a two-stage flowshop with m_2 parallel machines at the second stage, and b parallel intermediate storage units. A modified version of the algorithm of Johnson routinely produces solutions within 1% of optimal for instances with five jobs. This heuristic is incorporated into a branch and bound algorithm.

Sriskandarajah and Sethi (1989) describe a heuristic for $F_2|no\text{-}wait, m_1 = m_2 = \mu \geq 2|C_{max}$ that combines the list scheduling algorithm and the algorithm of Gilmore and Gomory, and provides an (attainable) worst case bound of $r = 3 - 1/\mu$. Although the exact worst case bound is not known, Sriskandarajah proves that, if the longest processing time (LPT) rule is used for list scheduling, then $7/3 - 2/3\mu \leq r \leq 8/3 - 2/3\mu$. These bounds are also valid for the blocking case.

It is well known (Fisher 1980) that heuristic algorithms often behave significantly better in practice than their worst cases guarantee. Therefore, it would be interesting to study the average case performance of the above algorithms under appropriate assumptions about the type of jobs that need to be processed in a typical shop.

3.8. $F_2|no\text{-}wait, ranked\ data\ |C_{max}$

Agnetis considers a variant of the makespan problem with two differences. First, the n jobs can be grouped into s lots, where within each lot all jobs are identical. Second, the decision maker knows an ordinal ranking for the lengths of operation times on each machine, but not the exact times. An $O(s \log s)$ time heuristic which uses an $s \times s$ transportation problem as a subroutine is designed. The relative error of the heuristic is bounded above by the ratio of the largest weight arc in the transportation problem to the optimal cost. From Lemma 4, the same procedure can be used for the analogous problem with blocking.

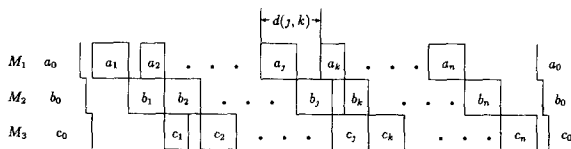


Figure 1. Equivalence of TSP and $F_3|no\text{-}wait|C_{max}$.

3.9. $F_m|no\text{-}wait|C_{max}$

Piehl (1960) shows the equivalence of this problem to a traveling salesman problem. Studies by Reddi and Ramamoorthy (1972, 1973a), Wismer (1972), and Goyal (1973) suggest heuristics designed for the latter problem. Grabowski and Syslo (1973) and Syslo (1974) consider variations of the problem and provide similar results. The following example shows this equivalence.

Example 3.2. Consider $m = 3$ and let $p_{1,j} = a_j, p_{2,j} = b_j, p_{3,j} = c_j, 1 \leq j \leq n$. Let $j = 0$ define a dummy job with $a_0 = b_0 = c_0 = 0$.

The traveling salesman intercity distance $d(j, k)$ in Figure 1 is the time duration between the start of job j on machine M_1 and the earliest time at which job k can start on that machine by respecting the no-wait restriction when k is scheduled next after j , where $d(j, k) = a_j + \max\{0, b_j - a_k, b_j + c_j - a_k - b_k\}$.

Pekny and Miller (1991) describe an m -machine, n -job no-wait flowshop problem arising in the batch manufacturing of specialty chemicals. They model this problem as a traveling salesman problem and describe exact and heuristic solution methods.

Das et al. (1990) study permutation schedules for $F_m|no\text{-}wait|C_{max}$, as well as for variants with finite intermediate storage. Four different simulated annealing algorithms are compared. The best results are provided by the algorithm of Metropolis et al., (1953) used with the Aarts and van Laarhoven (1985) annealing schedule.

A considerable body of literature exists on optimization procedures for the traveling salesman problem (Lawler et al. 1985). As a consequence of the transformation in Figure 1, these procedures can also be used to minimize makespan in the no-wait flowshop.

A local optimization heuristic, which repeatedly links (concatenates) jobs until a sequence is found, was proposed by King and Spachis. No worst case performance bounds are known for this approach.

An alternative heuristic approach was described by Röck and Schmidt (1983):

STEP 1. Convert the m machine flowshop problem into a two machine flowshop problem with processing times t_j^k for job j on machine k defined by

$$t_j^1 = \sum_{i=1}^{\lceil m/2 \rceil} p_{i,j}, \quad t_j^2 = \sum_{i=\lceil m/2 \rceil+1}^m p_{i,j}, \quad j = 1, \dots, n.$$

STEP 2. Solve this problem using the algorithm of Gilmore and Gomory. Stop.

The authors prove a tight worst case performance bound of $r = \lceil m/2 \rceil$ for this heuristic. If the problem to be solved contains a requirement that jobs maintain the same order on all machines, but with unlimited buffer, then the use of Johnson's (1954) algorithm at Step 2 is recommended. No performance bound is known for this heuristic when applied to the blocking case, and it would be interesting to pursue such research.

Assuming that the n jobs can be grouped into s lots each containing identical jobs, Agnetis proposes the heuristic solution of the problem using an $s \times s$ transportation problem. The worst case performance of this heuristic improves as the size of any lot grows, so it is asymptotically optimal with respect to lot size. If all lots have the same size $q = n/s$, the worst case performance ratio is $(m + 1)/2q$.

Panwalkar and Woollam (1979) prove useful results for specially structured processing time matrices in the $F_m|\text{no-wait}|C_{\max}$ problem. Let N denote the set of jobs, and M the set of machines. The conditions of the special structure, defining an "ordered" processing time matrix, are

- i. if $p_{t,j} > p_{t,k}$ for $j, k \in N$ and some $t \in M$, then $p_{i,j} \geq p_{i,k}$ for all $i \in M$.
- ii. if $p_{s,k} > p_{t,k}$ for some $k \in N$ and $s, t \in M$, then $p_{s,j} \geq p_{t,j}$ for all $j \in N$.

Given these conditions, let the jobs be numbered such that, for any $j < k$, $j, k \in N$, $p_{i,j} \leq p_{i,k}$ for all $i \in M$, then the sequence $n, \dots, 1$ minimizes $F_m|\text{no-wait}|C_{\max}$. Arora and Rana (1980) observe that without conditions ii, the case known as a "semi-ordered" processing time matrix, the distance matrix of the corresponding traveling salesman problem is a distribution matrix, i.e., $d(i, j) + d(r, s) \leq d(i, s) + d(r, j)$ for all $i < r$ and $j < s$. They show that an $O(n^2)$ algorithm can be devised for this case (see also Axsäter 1982). It has to be noted that the traveling salesman problem restricted to distribution matrices is solvable in $O(n^2)$ time by computing a shortest pyramidal tour (Lawler et al. 1985). Van der Veen and van Dal (1991) conjecture that $F_m|\text{no-wait}|C_{\max}$ is NP-complete, even if jobs have identical processing times on all but two machines, unless one of those machines is the first or last, or those machines are consecutive, in which cases polynomial algorithms are provided. Finally, a goal programming formulation that incorporates several objectives including makespan is given in Selen and Hott (1986), with examples and brief computational results.

3.10. $F_m|\text{blocking}|C_{\max}$

It is of interest to note that blocking problems also arise in production environments with limited intermediate buffers between machines, as well as with no buffer.

Levner (1969) provides a branch-and-bound approach for finding optimal solutions for this problem. The bounds on optimal value are derived from using the requirement of blocking to construct closed form expressions. Computational results are not provided, but the methodology has the potential to be effective.

Suhami and Mah (1981) incorporate a heuristic and a lower bounding procedure into a branch and bound algorithm for this problem. For instances with $m = 3$ and $6 \leq n \leq 10$, randomly generated schedules are typically 20% larger than optimal, but the branch-and-bound algorithm usually finds solutions within 1% of optimal very quickly.

3.11. $F_m|\text{no-wait}|\Sigma C_j$

Van Deman and Baker (1974) develop a variety of procedures for generating lower bounds on optimal value. These procedures are incorporated into a branch-and-bound algorithm which is capable of optimally solving instances up to $m = 6$ and $n = 12$.

Panwalkar and Woollam (1980) consider the ordered processing time matrix described in Section 3.9. The result here is exactly the opposite of that described for $F_m|\text{no-wait}|C_{\max}$. That is, if the jobs are numbered such that for any $j < k$, $j, k \in N$ and $p_{i,j} < p_{i,k}$ for all $i \in M$, then the sequence $1, \dots, n$ is optimal. Van der Veen and van Dal generalize these results to the case of semi-ordered processing time matrices. Related work by Adiri and Pohoryles (1982) shows that if later machines dominate earlier ones, that is, for $i < k$, $i, k \in M$, $\max_{1 \leq j \leq n} p_{i,j} \leq \min_{1 \leq j \leq n} p_{k,j}$, then scheduling the jobs in SPT order based on processing times on the last machine is optimal. Similarly, if earlier machines dominate later ones, or for $i > k$, $i, k \in M$, $\max_{1 \leq j \leq n} p_{i,j} \leq \min_{1 \leq j \leq n} p_{k,j}$, an optimal schedule can be found by using SPT ordering based on processing times on the first machine.

Rajendran and Chaudhuri (1990) describe and test two heuristics. One heuristic chooses jobs sequentially according to their total weighted processing time on all machines, with those weights being based on the positions of operations in the sequence of machines. This performs slightly better than a second heuristic that simply uses the sum of processing times for a job. For $12 \leq n \leq 20$, the average performance is considerably better than that of the heuristics of Bonney and Gundry (1976) and King and Spachis.

Gupta (1976) addresses the related weighted idle time problem, or $F_m|\text{no-wait}|\Sigma w_j I_j$. He implements Wismer's algorithm and compares its performance with that of a specially designed algorithm. The latter performed better, although within the same order of magnitude. Szwarc (1981) also addresses a variation of the weighted idle time problem similar to that considered by Gupta.

4. DETERMINISTIC JOBSHOP AND OPENSHP PROBLEMS

Here we review the literature on jobshop and openshop scheduling with no-wait and blocking.

4.1. $J_2|\text{no-wait}, p_{i,j} = 1|C_{\max}$

In this two-machine jobshop problem, a job consists of unit-time operations processed alternately on M_1 and M_2 . Since a job may be processed several times on a machine, we let K denote the total number of operations of the jobs. Kubiak (1989) suggests an $O(n^5 K)$ time algorithm that uses dynamic programming. A key feature is the decomposition of jobs into two subsets. Because each job alternates between machines, one subset takes up every alternate time unit on each machine, and the other subset follows it one unit of time later.

4.2. $J_m|no-wait|C_{max}$

The minimization of makespan in a no-wait jobshop is a very difficult problem computationally, and we are not aware of any attempts to solve large instances to optimality. Reddi and Ramamoorthy (1973b) derive a lower bound that is potentially useful for branch-and-bound, under a simple condition on the processing times, but do not give computational results.

Goyal (1975) observes that the problem of minimizing makespan in a no-wait flowshop is a traveling salesman problem (see Section 3.9). He therefore considers the same problem in a special jobshop environment as one of solving a sequence of traveling salesman problems. The lowest cost solution thus found is a heuristic solution to the problem.

Recently there have been significant advances in solution procedures for the general job shop problem with unlimited buffers (Adams et al. 1988), and it would be interesting to see whether similar ideas could be helpful in the no-wait and blocking cases.

4.3. $O|no-wait, p_{i,j} = 1|\Sigma C_j$

Here jobs are identical and each job consists of m unit time operations. The algorithm proposed by Adiri and Amit (1984) schedules m jobs without gaps in the first time interval $[0, m]$, the next m jobs in the second time interval $[m, 2m]$, and so on until the last r jobs go in the time interval $[(k-1)m, km]$, where k is the largest integer such that $k = \lceil n/m \rceil$ and $r = km - n$. The optimal sum of completion times is $m(k+1)(mk+2r)/2$, and the complexity of this algorithm is $O(mn)$.

4.4. $O|no-wait, p_{i,j} = 1|\Sigma w_j U_j$

Kubiak et al. (1991) suggest the following algorithm.

Algorithm

STEP 1. $L = \emptyset$, the set of late jobs. Order the jobs such that $w_1 \geq \dots \geq w_n$.

STEP 2. Take the smallest index unscheduled job j . Find the largest integer t such that $tm \leq d_j$, and such that there exists at least one available position in the interval $[(t-1)m, tm]$ on each machine. If possible, schedule job j in that interval. Otherwise, set $L \leftarrow L \cup \{j\}$.

STEP 3. Repeat Step 2 until all jobs are either scheduled or put into L .

STEP 4. Fill all empty positions with late jobs L in the interval $[(r-1)m, rm]$, for $r = 1, \dots, k$, where $k = \lceil n/m \rceil$. Stop.

By keeping an ordered list of empty positions for each d_j , the algorithm can be implemented in $O(n \log n)$ time. Note that if job j is late in the schedule constructed by the algorithm, then all the positions before its due date are already occupied by jobs with larger weights. It follows that the algorithm is optimal.

4.5. Openshop Equivalence Results

Brucker et al. (1993) prove the equivalence of several no-wait openshop problems with unit processing times to problems in a parallel machine environment. This permits the use of polynomial time algorithms which are available for the latter problems. A similar equivalence result shows that problems $O|no-wait, p_{i,j} = 1, prec|C_{max}$, $O|no-wait, p_{i,j} = 1, out-tree|L_{max}$ and $O|no-wait, p_{i,j} = 1, prec|\Sigma C_j$ are NP -complete. All these results are summarized in Table I.

5. STOCHASTIC PROBLEMS

Several surveys on stochastic scheduling are available (Pinedo and Schrage 1982; Forst 1984; Möhring et al. 1984, 1985; and Möhring and Radermacher 1985).

5.1. $F_2|blocking|E(C_{max})$ —Minimize Expected Makespan

Pinedo and Schrage (1982) show that the general version of this problem is equivalent to a deterministic traveling salesman problem. Consider a special case in which the processing times of a job on the different machines are i.i.d., and the distributions of processing times of jobs satisfy $P\{p_{i,j} > t\} \geq P\{p_{i,k} > t\}$ for $j < k$, $i = 1, 2$. Then, if n is even, job sequence $n, n-2, \dots, 4, 2, 1, 3, \dots, n-3, n-1$ and the reverse of that sequence are both optimal. If n is odd, sequence $n, n-2, \dots, 3, 1, 2, 4, \dots, n-3, n-1$ and its reverse are both optimal. Similar results are derived for the case where the expected processing times of all jobs are equal, but the variances of processing times are different.

5.2. $F_m|blocking|E(C_{max})$ —Stochastically Minimize Makespan

Note that a sequence that stochastically minimizes the makespan not only minimizes the expected makespan, but also any moment of the makespan. Two jobs j, k are non-overlapping if the probability $P\{p_{i,j} \geq p_{i,k}\}$ is either 0 or 1, where $p_{i,j}, p_{i,k}$ are the random variables representing their processing times. For n nonoverlapping jobs, each having i.i.d. processing times for operations, Pinedo (1982) shows that an SPT-LPT schedule minimizes the expected makespan. Note that a schedule of jobs $j_1, \dots, j_n$ is an SPT-LPT schedule if there exists a job j_k such that $E(p_{i,j_1}) \leq \dots \leq E(p_{i,j_k})$ and $E(p_{i,j_k}) \geq \dots \geq E(p_{i,j_n})$ for $i = 1, 2$, where $E(p_{i,j})$ is the expected processing time of job j on machine i .

Foley and Suresh (1984, 1986) consider stochastically minimizing the makespan. They obtain optimal schedules for four cases of processing times: (1) n nonoverlapping jobs, (2) n nonoverlapping jobs and one arbitrary job, (3) n stochastically identical jobs and two shorter jobs, and (4) n nonoverlapping jobs and two arbitrary jobs. The generalization of these no-buffer results to the finite buffer case is apparently not straightforward.

Kijima et al. (1990) show for cases (1) and (2) above that adding any size of intermediate buffers to the flowshops with these schedules does not reduce the makespan,

and that these schedules are optimal for the infinite buffer case. For cases (3) and (4), such direct extensions do not exist, because adding buffers may improve the makespan. However, they also show that the above result for case (3) can be extended to the finite buffer environment, whereas the results for case (4) can be extended in a slightly restricted form. The above results for finite buffers are also valid for the infinite buffer case.

6. CONCLUSIONS AND SUGGESTIONS FOR FUTURE RESEARCH

The quality and quantity of the work which we have described suggest that the class of no-wait and blocking machine scheduling problems, together with related problems at its boundary, represents an important recent direction of research. This importance is likely to increase because of the variety of no-wait and blocking scheduling applications arising in modern manufacturing. We discuss many of those applications in detail. We present the first detailed summary of computational complexity results in this area, and note that several problems have still unresolved complexity. We then describe algorithms and heuristics for several deterministic flowshop, jobshop, and openshop problems.

There is room for much research progress with respect to flowshop makespan problems. No-wait or blocking jobshops have also received surprisingly little attention from either a theoretical or computational perspective. Two important open issues are the extent to which ideas from a traditional scheduling environment can be useful for no-wait jobshop scheduling, and the usefulness of general heuristic methodologies such as simulated annealing and tabu search in providing good quality solutions to such problems.

Results for no-wait scheduling problems with stochastic processing times are limited in scope. It would be unreasonable to expect such results to exceed those derived for a traditional scheduling environment, because of the apparently greater difficulty imposed by the no-wait assumption. However, it may be possible to derive results for the problem of maximizing the probability of schedule completion by a given time.

Variations of the no-wait scheduling environment, especially those involving blocking and finite intermediate storage, model numerous applications. It is presently unclear to what extent such problems may be computationally tractable through mathematical programming techniques such as Lagrangean relaxation and cutting planes. The structure of those problems suggests that they may be amenable to such approaches. In conclusion, we believe that no-wait and blocking scheduling will attract increasing attention from researchers, as the applicability of such models to modern manufacturing problems becomes clearer, and as the variety of exciting related research issues unfolds.

ACKNOWLEDGMENTS

We are grateful to Marc Posner (The Ohio State University) for comments on an initial draft of this manuscript. We also wish to thank two anonymous referees, whose detailed comments on earlier drafts greatly improved the paper. This research was supported by the Summer Fellowship Program (Fisher College of Business, The Ohio State University), NSERC Grant OGP0104900 and a Connaught Grant (University of Toronto).

REFERENCES

- AARTS, E. H. L., AND P. J. M. VAN LAARHOVEN. 1985. Statistical Cooling: A General Approach to Combinatorial Optimization Problems. *Phillips J. Res.* **40**, 193.
- ADAMS, J., E. BALAS, AND D. ZAWACK. 1988. The Shifting Bottleneck Procedure for Job Shop Scheduling. *Mgmt. Sci.* **34**, 391–401.
- ADIRI, I., AND N. AMIT. 1984. Openshop and Flowshop Scheduling to Minimize Sum of Completion Times. *Computers and Opns. Res.* **11**, 275–284.
- ADIRI, I., AND D. POHORYLES. 1982. Flowshop/No-idle or No-wait Scheduling to Minimize the Sum of Completion Times. *Naval Res. Logistics Quarterly.* **29**, 3, 495–504.
- AGNETIS, A. 1989. No-wait Flow Shop Scheduling with Large Lot Size. Research Report 16.89, Dipartimento di Informatica e Sistemistica, Università di Roma “La Sapienza”, Rome.
- ARORA, R. K., AND S. P. RANA. 1980. Scheduling in a Semi-ordered Flowshop without Intermediate Queues. *AIIE Trans.* **12**, 263–272.
- AXSÄTER, S. 1982. On Scheduling in a Semi-ordered Flowshop without Intermediate Queues. *IIE Trans.* **14**, 128–130.
- BAKER, K. R. 1974. *Introduction to Sequencing and Scheduling*. Addison-Wesley, Reading, MA.
- BLAZEWICZ, J., W. KUBIAK, AND J. SZWARCFTER. 1988. Scheduling Unit-time Tasks on Flow-shops under Resource Constraints. *Annals of Opns. Res.* **16**, 255–266.
- BLAZEWICZ, J., J. K. LENSTRA, AND A. H. G. RINNOOY KAN. 1983. Scheduling Subject to Resource Constraints: Classification and Complexity. *Discrete Applied Math.* **5**, 11–24.
- BONNEY, M. C., AND S. W. GUNDRY. 1976. Solutions to the Constrained Flowshop Sequencing Problem. *Opnl. Res. Quarterly.* **24**, 869–883.
- BRUCKER, P., M. R. GAREY, AND D. S. JOHNSON. 1977. Scheduling Equal-length Tasks under Tree-like Precedence Constraints to Minimize Maximum Lateness. *Math. O. R.* **2**, 275–284.
- BRUCKER, P., B. JURISCH, AND M. JURISCH. 1993. Openshop Problems with Unit Time Operations. *Zeitschrift für Opns. Res.* **37**, 1, 59–73.
- CALLAHAN, J. R. 1971. The Nothing Hot Delay Problems in the Production of Steel, Ph.D. Thesis, Department of Industrial Engineering, University of Toronto, Canada.
- COFFMAN, E. G., JR. 1976. *Computer and Job-Shop Scheduling Theory*, Wiley, New York.
- COFFMAN, E. G., JR., AND R. L. GRAHAM. 1972. Optimal Scheduling for Two-processor Systems. *Acta Informatica.* **1**, 200–213.

- CONWAY, R. W., W. MAXWELL, J. O. MCCLAIN, AND L. J. THOMAS. 1988. The Role of Work-in-process Inventory in Serial Production Lines. *Opns. Res.* **36**, 229–241.
- CONWAY, R. W., W. L. MAXWELL, AND L. W. MILLER. 1967. *Theory of Scheduling*, Addison-Wesley, Reading, MA.
- DAS, H., P. T. CUMMINGS, AND M. D. LE VAN. 1990. Scheduling of Serial Multiproduct Batch Processes via Simulated Annealing. *Comp. Chem. Eng.* **14**, 1351–1362.
- DUDEK, R. A., S. S. PANWALKAR, AND M. L. SMITH. 1992. The Lessons of Flowshop Scheduling Research. *Opns. Res.* **40**, 7–13.
- DUTTA, S. K., AND A. A. CUNNINGHAM. 1975. Sequencing Two-machine Flow-shops with Finite Intermediate Storage. *Mgmt. Sci.* **21**, 989–996.
- EGLI, U. M., AND D. W. T. RIPPIN. 1981. Short-term Scheduling for Multi-product Batch Chemical Plants. *American Institute of Chemical Engineers Meeting*, Houston.
- FISHER, M. L. 1980. Worst-case Analysis of Heuristic Algorithms. *Mgmt. Sci.* **26**, 1–17.
- FOLEY, R., AND S. SURESH. 1984. Stochastically Minimizing the Makespan in Flow Shops. *Naval Res. Logistics Quarterly*. **31**, 551–557.
- FOLEY, R., AND S. SURESH. 1986. Scheduling n Nonoverlapping Jobs and Two Stochastic Jobs in a Flowshop. *Naval Res. Logistics Quarterly*. **33**, 123–128.
- FORST, F. G. 1984. A Review of the Static Stochastic Job Sequencing Literature. *Opsearch*. **21**, 3, 127–144.
- FRENCH, S. 1982. *Sequencing and Scheduling*. Wiley, New York.
- GAREY, M. R., AND D. S. JOHNSON. 1976. Scheduling Tasks with Nonuniform Deadlines on Two Processors. *J. Assoc. Comput. Machinery*. **23**, 461–467.
- GAREY, M. R., AND D. S. JOHNSON. 1977. Two-processor Scheduling with Start-times and Deadlines. *SIAM J. Comput.* **6**, 416–426.
- GAREY, M. R., AND D. S. JOHNSON. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, San Francisco.
- GILMORE, P. C., AND R. E. GOMORY. 1964. Sequencing a One State-variable Machine: A Solvable Case of the Traveling Salesman Problem. *Opns. Res.* **12**, 655–679.
- GLOVER, F. 1990. Tabu Search: A Tutorial. *Interfaces*. **20**, 4, 74–94.
- GONZALEZ, M. J., JR. 1977. Deterministic Processor Scheduling. *Comput. Surveys*. **9**, 173–204.
- GONZALEZ, T. 1982. Unit Execution Time Shop Problems. *Math. O. R.* **7**, 57–66.
- GONZALEZ, T., AND S. SAHNI. 1976. Openshop Scheduling to Minimize Finish Time. *J. Assoc. Comput. Machinery*. **23**, 665–679.
- GOYAL, S. K. 1973. A Note on the Paper: On the Flowshop Sequencing Problem with No-wait in Process. *Opl. Res. Quarterly*. **24**, 130–133.
- GOYAL, S. K. 1975. Jobshop Sequencing Problem with No-wait in Process. *International J. Production Res.* **13**, 197–206.
- GOYAL, S. K., AND C. SRISKANDARAJAH. 1988. No-wait Shop Scheduling: Computational Complexity and Approximate Algorithms. *Opsearch*. **25**, 220–244.
- GRABOWSKI, J., AND M. M. SYSLO. 1973. On Some Machine Sequencing Problems (1). *Zastosowania Matematyki*. **13**, 340–345.
- GRAHAM, R. L., E. L. LAWLER, J. K. LENSTRA, AND A. H. G. RINNOOY KAN. 1979. Optimization and Approximation in Deterministic Sequencing and Scheduling: A Survey. *Annals of Discrete Math.* **5**, 287–326.
- GUPTA, J. N. D. 1976. Optimal Flowshop Schedules with No Intermediate Storage Space. *Naval Res. Logistics Quarterly*. **23**, 235–243.
- HALL, N. G., H. KAMOUN, AND C. SRISKANDARAJAH. 1995. Scheduling in Robotic Cells: Complexity and Steady State Analysis. Working paper 95-21, Department of Industrial Engineering, University of Toronto.
- HALL, N. G., H. KAMOUN, AND C. SRISKANDARAJAH. 1996. Scheduling in Robotic Cells: Classification, Two and Three Machine Cells. *Opns. Res.*, to appear.
- HALL, R. W. 1988. Cyclic Scheduling for Improvement. *International J. Production Res.* **26**, 3, 457–472.
- HEFETZ, N., AND I. ADIRI. 1982. A Note on the Influence of Missing Operations on Scheduling Problems. *Naval Res. Logistics Quarterly*. **29**, 3, 535–539.
- HSU, E., AND M. STEIN. 1991. Anodize Load Scheduling for Diamond Aluminum. Undergraduate Thesis, Department of Industrial Engineering, University of Toronto, Canada.
- HU, T. C. 1961. Parallel Sequencing and Assembly Line Problems. *Opns. Res.* **9**, 841–848.
- JACKSON, J. R. 1955. Scheduling a Production Line to Minimize Maximum Tardiness, Research Report 43, Management Science Research Project, University of California, Los Angeles.
- JACKSON, J. R. 1956. An Extension of Johnson's Results on Job Lot Scheduling. *Naval Res. Logistics Quarterly*. **3**, 201–203.
- JOHNSON, S. M. 1954. Optimal Two- and Three-stage Production Schedules with Setup Times Included. *Naval Res. Logistics Quarterly*. **1**, 61–68.
- KAMOUN, H., N. G. HALL, AND C. SRISKANDARAJAH. 1993. Scheduling in Robotic Cells: Heuristics and Cell Design, Working Paper 93-08, Department of Industrial Engineering, University of Toronto.
- KAMOUN, H., AND C. SRISKANDARAJAH. 1993. The Complexity of Scheduling Jobs in Repetitive Manufacturing Systems. *European J. Opnl. Res.* **70**, 3, 350–364.
- KIJIMA, M., N. MAKIMOTO, AND H. SHIRAKAWA. 1990. Stochastic Minimization of the Makespan in Flowshops with Identical Machines and Buffers of Arbitrary Size. *Opns. Res.* **38**, 924–928.
- KING, J. R., AND A. S. SPACHIS. 1980. Heuristics for Flowshop Scheduling. *International J. Production Res.* **18**, 3, 345–357.
- KISE, H., T. SHIOYAMA, AND T. IBARAKI. 1991. Automated Two-machine Flowshop Scheduling: A Solvable Case. *IIE Trans.* **23**, 1, 10–16.
- KNOPF, F. C. 1985. Sequencing a Generalized Two-stage Flowshop with Finite Intermediate Storage. *Computers and Chemical Eng.* **9**, 207–221.
- KUBIAK, W. 1989. A Pseudo-polynomial Algorithm for a Two-machine No-wait Jobshop Scheduling Problem. *European J. Opnl. Res.* **43**, 267–270.
- KUBIAK, W., C. SRISKANDARAJAH, AND K. ZARAS. 1991. A Note on the Complexity of Openshop Scheduling Problems. *INFOR*. **29**, 4, 284–294.

- KURIYAN, K. 1987. Scheduling of Batch Processes. Ph.D. Thesis, School of Chemical Engineering, Purdue University, West Lafayette, Indiana.
- KURIYAN, K., AND G. V. REKLAITIS. 1985. Approximate Scheduling Algorithms for Network Flowshops. *The Institute of Chemical Engineers, Symposium Series* **92**, 79–90.
- KURIYAN, K., AND G. V. REKLAITIS. 1989. Sequencing Network Flowshops so as to Minimize Makespan. *Computers and Chemical Eng.* **13**, 187–200.
- LAWLER, E. L. 1976. Sequencing to Minimize the Weighted Number of Tardy Jobs. *Revue Française d'Automatique, d'Informatique et de Recherche Opérationnelle*. **10** (Supplement 5), 27–33.
- LAWLER, E. L. 1982. Recent Results in the Theory of Machine Scheduling. In *Mathematical Programming: The State of the Art*, A. Bachem, M. Grötschel and B. Korte (eds.). Springer, 202–234.
- LAWLER, E. L., J. K. LENSTRA, AND A. H. G. RINNOOY KAN. 1982. Recent Developments in Deterministic Sequencing and Scheduling: A Survey. In *Deterministic and Stochastic Scheduling*, M. A. H. Dempster, J. K. Lenstra, and A. H. G. Rinnooy Kan (eds.). Dordrecht, 35–73.
- LAWLER, E. L., J. K. LENSTRA, A. H. G. RINNOOY KAN, AND D. B. SHMOYS. 1985. *The Traveling Salesman Problem*. Wiley, New York.
- LAWLER, E. L., J. K. LENSTRA, A. H. G. RINNOOY KAN, AND D. B. SHMOYS. 1989. Sequencing and Scheduling: Algorithms and Complexity. Report BS-R8909. Center for Mathematics and Computer Science, Amsterdam, The Netherlands.
- LEISTEN, R. 1990. Flow Shop Sequencing Problems with Limited Buffer Storage. *International J. Production Res.* **28**, 11, 2085–2100.
- LENSTRA, J. K. 1977. *Sequencing by Enumerative Methods*. Mathematical Centre Tracts 69, Mathematisch Centrum, Amsterdam.
- LENSTRA, J. K., AND A. H. G. RINNOOY KAN. 1978. Complexity of Scheduling under Precedence Constraints. *Opns. Res.* **26**, 22–35.
- LENSTRA, J. K., AND A. H. G. RINNOOY KAN. 1979. Computational Complexity of Discrete Optimization Problems. *Annals of Discrete Math.* **4**, 121–140.
- LENSTRA, J. K., AND A. H. G. RINNOOY KAN. 1985. Sequencing and Scheduling. In *Combinatorial Optimization: Annotated Bibliographies*, M. O'Hegartaigh, J. K. Lenstra and A. H. G. Rinnooy Kan (eds.), Wiley, Chichester, 164–189.
- LENSTRA, J. K., A. H. G. RINNOOY KAN, AND P. BRUCKER. 1977. Complexity of Machine Scheduling Problems. *Annals of Discrete Math.* **1**, 343–362.
- LEVNER, E. V. 1969. Optimal Planning of Parts Machining on a Number of Machines. *Automatic Remote Control*. **12**, 1972–1978.
- LOGENDRAN, R., AND C. SRISKANDARAJAH. 1996. Sequencing of Robot Activities and Parts in Two Machine Robotic Cells. *Int. J. Prod. Res.*, to appear.
- LOGENDRAN, R., AND C. SRISKANDARAJAH. 1993. Two-machine Group Scheduling Problem with Blocking and Anticipatory Setups. *European J. Opnl. Res.*, **69**, 3, 467–481.
- MATSUO, H. 1990. Cyclic Sequencing Problems in a Two Machine Permutation Flowshop: Complexity, Worst Case and Average Case Analysis. *Naval Res. Logistics*. **37**, 679–694.
- MCCORMICK, S. T., M. L. PINEDO, S. SHENKER, AND B. WOLF. 1989. Sequencing in an Assembly Line with Blocking to Minimize Cycle Time. *Opns. Res.* **37**, 925–935.
- MCCORMICK, S. T., M. L. PINEDO, AND B. WOLF. 1987. The Complexity of Scheduling a Flexible Assembly Line with Finite Buffers. IE/OR Technical Report. Columbia University, New York.
- METROPOLIS, N., A. ROSENBLUTH, M. ROSENBLUTH, A. TELLER, AND E. TELLER. 1953. Equation of State Calculations by Fast Computing Machines. *J. Chem. Physics*. **21**, 1087.
- MÖHRING, R. H., AND F. J. RADERMACHER. 1985. Introduction to Stochastic Scheduling Problems. In *Contributions to Operations Research, Proceedings of the Oberwolfach Conference on Operations Research*, K. Neumann and D. Pallaschke (eds.). Springer-Verlag, Heidelberg, 72–130.
- MÖHRING, R. H., F. J. RADERMACHER, AND G. WEISS. 1984. Stochastic Scheduling Problems, I: General Strategies. *Zeitschrift für Opns. Res.* **28**, 193–260.
- MÖHRING, R. H., F. J. RADERMACHER, AND G. WEISS. 1985. Stochastic Scheduling Problems, II: Set Strategies. *Zeitschrift für Opns. Res.* **29**, 65–104.
- MONMA, C. L. 1979. The Two Machine Maximum Flow-time Problem with Series Parallel Precedence Constraints: An Algorithm and Extensions. *Opns. Res.* **25**, 792–798.
- NAWAZ, M., E. E. ENSCORE, AND I. HAM. 1983. A Heuristic Algorithm for the m -machine, n -job Flow-shop Sequencing Problem. *Omega*. **11**, 91–95.
- PANWALKAR, S. S., AND C. R. WOOLLAM. 1979. Flowshop Scheduling Problem with No In-process Waiting: A Special Case. *J. Opns. Res. Soc.* **30**, 661–664.
- PANWALKAR, S. S., AND C. R. WOOLLAM. 1980. Ordered Flowshop Problems with No In-process Waiting: Further Results. *J. Opnl. Res. Soc.* **31**, 1039–1043.
- PAPADIMITRIOU, C., AND P. KANELLAKIS. 1980. Flowshop Scheduling with Limited Temporary Storage. *J. Assoc. Comput. Machinery*. **27**, 533–549.
- PEKNY, J. F., AND D. L. MILLER. 1991. Exact Solution of the No-wait Flowshop Scheduling Problem with a Comparison to Heuristic Methods. *Computers and Chem. Eng.* **15**, 741–748.
- PIEHLER, J. 1960. Ein Beitrag zum Reihenfolgeproblem. *Unternehmensforschung*. **4**, 138–142.
- PINEDO, M. 1982. Minimizing the expected makespan in stochastic flowshops. *Opns. Res.* **30**, 148–162.
- PINEDO, M., AND L. SCHRAGE. 1982. Stochastic Shop Scheduling: A Survey. In *Deterministic and Stochastic Scheduling*, M. A. H. Dempster, J. K. Lenstra and A. H. G. Rinnooy Kan (eds.). Dordrecht, 181–196.
- RAJENDRAN, C., AND D. CHAUDHURI. 1990. Heuristic Algorithms for Continuous Flow-shop Problem. *Naval Res. Logistics*. **37**, 695–705.
- RAMUDHIN, A., AND H. D. RATLIFF. 1992a. On the Complexity of the Process Shop. Working Paper 92-10, Faculté des Sciences de L'Administration, Université Laval, Québec, Canada.
- RAMUDHIN, A., AND H. D. RATLIFF. 1992b. Generating Daily Production Schedules in Process Industries. Working Paper 92-51, Faculté des Sciences de L'Administration, Université Laval, Québec, Canada.

- REDDI, S. S., AND C. V. RAMAMOORTHY. 1972. On the Flowshop Sequencing Problem with No-wait in Process. *Opnl. Res. Quarterly*. **23**, 323–331.
- REDDI, S. S., AND C. V. RAMAMOORTHY. 1973a. Reply to Dr. Goyal's Comments. *Opnl. Res. Quarterly*. **24**, 133–134.
- REDDI, S. S., AND C. V. RAMAMOORTHY. 1973b. A Scheduling Problem. *Opnl. Res. Quarterly*. **24**, 441–446.
- REKLAITIS, G. V. 1982. Review of Scheduling of Process Operations. *AIChE Symposium Series*. **78**, 119–133.
- RINNOOY KAN, A. H. G. 1976. *Machine Scheduling Problems: Classification, Complexity and Computations*. Nijhoff, The Hague.
- RÖCK, H. 1983. Scheduling Unit Task Shops with Resource Constraints and Excess Usage Costs. Report 83-21, Fachbereich Informatik, Technical University of Berlin, Berlin (FRG).
- RÖCK, H. 1984a. The Three Machine No-wait Flowshop Problem Is NP-complete. *J. Assoc. Comput. Machinery*. **31**, 336–345.
- RÖCK, H. 1984b. Some New Results in No-wait Flowshop Scheduling. *Zeitschrift für Opns. Res.* **28**, 1–16.
- RÖCK, H., AND G. SCHMIDT. 1983. Machine Aggregation Heuristics in Shop-scheduling. *Methods Opns. Res.* **45**, 303–314.
- SAHNI, S., AND Y. CHO. 1979. Complexity of Scheduling Shops with No-wait In Process. *Math. O. R.* **4**, 448–457.
- SALVADOR, M. S. 1973. A Solution of a Special Class of Flowshop Scheduling Problems. In *Proceedings of the Symposium on the Theory of Scheduling and Its Applications*. Springer-Verlag, Berlin, 83–91.
- SCHONBERGER, R. J. 1982. *Japanese Manufacturing Techniques: Nine Hidden Lessons in Simplicity*. The Free Press, New York.
- SELEN, W. J., AND D. D. HOTT. 1986. A New Formulation and Solution of the Flowshop Scheduling Problem with No In Process Waiting. *App. Math. Modelling*. **10**, 246–248.
- SETHI, R. 1976. Algorithms for Minimal Length Schedules. In *Computer and Job-Shop Scheduling Theory*, E. G. Coffman, Jr. (ed). Wiley, New York, 51–99.
- SETHI, S. P., C. SRISKANDARAJAH, J. BLAZEWCZ, G. SORGER, AND W. KUBIAK. 1992. Sequencing of Robot Moves and Multiple Parts in a Robotic Cell. *International J. Flexible Manufacturing Sys.* **4**, 331–358.
- SIMONS, B., AND M. K. WARMUTH. 1989. A Fast Algorithm for Multiprocessor Scheduling of Unit Length Jobs. *SIAM J. Comput.* **18**, 4, 690–710.
- SRISKANDARAJAH, C. 1993. Performance of Scheduling Algorithms for No-wait Flowshops with Parallel Machines. *European J. Opnl. Res.* **70**, 3, 365–378.
- SRISKANDARAJAH, C., AND S. K. GOYAL. 1989. Scheduling of a Two-machine Flowshop with Processing Time Linearly Dependent on Job Waiting Time. *J. Opnl. Res. Soc.* **40**, 10, 907–921.
- SRISKANDARAJAH, C., N. G. HALL AND H. KAMOUN. 1996. Scheduling Large Robotic Cells. Working paper. Fisher College of Business, The Ohio State University.
- SRISKANDARAJAH, C., AND P. LADET. 1986. Some No-wait Shops Scheduling Problems: Complexity Aspects. *European J. Opnl. Res.* **24**, 3, 424–438.
- SRISKANDARAJAH, C., AND S. P. SETHI. 1989. Scheduling Algorithms for Flexible Flowshops: Worst and Average Case Performance. *European J. Opnl. Res.* **43**, 2, 143–160.
- SRISKANDARAJAH, C., AND E. WAGNEUR. 1991. Hierarchical Control of the Two Processor Flow-shop with State Dependent Processing Times: Complexity Analysis and Approximate Algorithms. *INFOR*. **29**, 3, 193–205.
- STRUSEVICH, V. A. 1990. Two Machine Flow Shop No-wait Scheduling Problem with Setup, Processing and Removal Times Separated. Report 9094/A, Econometric Institute, Erasmus University, Rotterdam.
- STRUSEVICH, V. A. 1995. Two Machine Flow Shop Scheduling Problem with No Wait In Process: Controllable Machine Speeds. *Discrete Appl. Math.* **59**, 75–86.
- STRUSEVICH, V. A. 1991. Complexity Aspects of Shop Scheduling Problems. Doctoral Thesis, Erasmus University, Rotterdam.
- STRUSEVICH, V. A., AND C. M. ZWANEVELD. 1991. Complexity of the Two-stage No-wait Assembly Problem with Setup and Removal Times Separated, Report 9142/A, Econometric Institute, Erasmus University, Rotterdam.
- SUHAMI, I., AND R. S. H. MAH. 1981. An Implicit Enumeration Scheme for the Flowshop Problem with No Intermediate Storage. *Computers and Chem. Eng.* **5**, 83–91.
- SYSLO, M. M. 1974. On Some Machine Sequencing Problems (II). *Zastosowania Matematyki*. **14**, 93–97.
- SZWARC, W. 1981. A Note on the Flow-shop Problem without Interruptions in Job Processing. *Naval Res. Logistics Quarterly*. **28**, 4, 665–669.
- ULLMAN, J. D. 1975. NP-Complete Scheduling Problems. *J. Computer and System Sci.* **10**, 384–393.
- VAN DEMAN, J. M., AND K. R. BAKER. 1974. Minimizing Mean Flow Time in the Flowshop with No Intermediate Queues. *AIIE Trans.* **6**, 28–34.
- VAN DER VEEN, J. A. A., AND R. VAN DAL. 1991. Solvable Cases of the No-wait Flow-shop Scheduling Problem. *J. Opnl. Res. Soc.* **42**, 11, 971–980.
- WAGNEUR, E., AND C. SRISKANDARAJAH. 1990. Control and Scheduling of m -processor Flowshop with State Dependent Processing Times. *Cahier du GERAD G-90-28*, École des Hautes Études Commerciales, Montréal, Canada.
- WAGNEUR, E., AND C. SRISKANDARAJAH. 1993a. The Two-machine Permutation Flow Shop with State Dependent Processing Times. *Naval Res. Logistics*. **40**, 697–717.
- WAGNEUR, E., AND C. SRISKANDARAJAH. 1993b. Optimal Control of a Class of DEDS: Flowshop with State Dependent Processing Times. *Discrete Event Dynamic Systems: Theory and Applications*. **3**, 397–425.
- WISMER, D. A. 1972. Solution of the Flowshop Scheduling Problem with No Intermediate Queues. *Opns. Res.* **20**, 689–697.
- ZANGWILL, W. I. 1987. From EOQ Towards ZI. *Mgmt. Sci.* **33**, 10, 1209–1223.